

Sparse Table :

```

/*
SparseTable sp(a, [](int a, int b) {
    return max(a, b);
});
*/
template<typename T, class CMP = function<T(const T &, const T &>>
class SparseTable {
public:
    int n;
    vector<vector<T>> sp;
    CMP func;
    SparseTable(const vector<T> &a, const CMP &f) : func(f) {
        n = a.size();
        int max_log = 32 - __builtin_clz(n);
        sp.resize(max_log);
        sp[0] = a;
        for (int j = 1; j < max_log; ++j) {
            sp[j].resize(n - (1 << j) + 1);
            for (int i = 0; i + (1 << j) <= n; ++i) {
                sp[j][i] = func(
                    sp[j - 1][i],
                    sp[j - 1][i + (1 << (j - 1))])
            };
        }
    }
    T get(int l, int r) const {
        int lg = __lg(r - l + 1);
        return func(
            sp[lg][l],
            sp[lg][r - (1 << lg) + 1]
        );
    }
};

```

Build Segment Tree :

```

void build (int l , int r , int node , vector<int> & arr){
    if(l==r){
        if(l<arr.size()){
            seg[node] = arr[l];
        }
        return;
    }
}

```

```

    }
    build(l,mid,L,arr);
    build(mid+1,r,R,arr);
    seg[node] = merge (seg[L],seg[R]);
}

```

Segment Tree :

```

struct Segtree
{
    #define L 2*node+1
    #define R 2*node+2
    #define mid ((l+r)/2)

    vector<int> seg ;
    int sz, skip ;

    int merge (int a ,int b){
        return a+b ;
    }

    int query (int l , int r , int node ,int lx , int rx){
        if(r<lx || rx<l)return skip ;
        if(l>=lx&&r<=rx) return seg[node];
        int left = query (l , mid ,L , lx , rx);
        int right= query (mid+1,r,R , lx,rx);
        return merge(left,right);
    }

    void update (int l , int r, int node , int indx , int val){
        if(l==r){
            seg[node]=val;
            return ;
        }
        if(indx<=mid){
            update(l,mid,L,indx,val);
        }
        else{
            update(mid+1,r,R,indx,val);
        }
        seg[node] = merge (seg[L],seg[R]);
    }
}

Segtree(int szz){
    int n = szz;
    sz = 1 ;
    skip = 0 ;
    while (sz<n) sz*=2 ;
    seg = vector<int> (sz*2 , skip);
}

```

```

int queryCall(int l , int r){
    return query(0,sz-1,0,l,r);
}
void updateCall(int val , int indx){
    update(0,sz-1,0,indx,val);
}

#undef L
#undef R
#undef mid
};

```

Lazy Segment Tree :

```

struct Segtree
{
#define L 2*node+1
#define R 2*node+2
#define mid ((l+r)/2)

vector<int> seg , typ , lazy;
int sz, skip ;

int merge (int a ,int b){
    return a+b ;
}

void propagate (int l , int r , int node){
    if(typ[node]==-1)return ;
    if(l!=r){
        lazy[L] += lazy[node];
        lazy[R] += lazy[node];
        typ[L] = typ[node] ;
        typ[R] = typ[node] ;
    }
    seg[node] += lazy[node]*(r-l+1);
    lazy[node] = 0 ;
    typ[node] = -1 ;
}

int query (int l , int r , int node ,int lx , int rx){
    propagate(l,r,node);
    if(r<lx || rx<l)return skip ;
    if(l>=lx&&r<=rx) return seg[node];
    int left = query (l , mid ,L , lx , rx);
    int right= query (mid+1,r,R , lx,rx);
    return merge(left,right);
}

void update (int l , int r , int node , int lx , int rx , int val){
    propagate(l,r,node);
    if(r<lx || rx<l)return ;

```

```

        if(l>=lx&&r<=rx) {
            lazy [node] += val ;
            typ [node] = 1;
            propagate(l,r,node);
            return ;
        }
        update(l,mid,L,lx,rx,val);
        update(mid+1,r,R,lx,rx,val);
        seg[node] = merge (seg[L],seg[R]);
    }

    Segtree(int sz){
        int n = sz;
        sz = 1 ;
        skip = 0 ;
        while (sz<n) sz*=2 ;
        seg = vector<int> (sz*2 , skip);
        lazy = vector<int>(sz*2 , skip) ;
        typ = vector<int>(sz*2 , -1) ;
    }

    int queryCall(int l , int r){
        return query(0,sz-1,0,l,r);
    }
    void updateCall(int l , int r , int val){
        update(0,sz-1,0,l,r,val);
    }

    #undef L
    #undef R
    #undef mid
};

```

BIT Point Update Range Query :

```

struct BIT {
    int n;
    vector<int> bit;

    BIT(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void add(int i, int val) {
        ++i;
        while (i <= n) {
            bit[i] += val;
            i += i & -i;
        }
    }
};

```

```

    }
}

int prefixQuery(int i) {
    ++i;
    int res = 0;
    while (i > 0) {
        res += bit[i];
        i -= i & -i;
    }
    return res;
}

int rangeQuery(int l, int r) {
    return prefixQuery(r) - prefixQuery(l - 1);
}
};

```

BIT Range Update Point Query :

```

struct BIT {
    int n;
    vector<int> bit;

    BIT(int _n) {
        n = _n;
        bit.assign(n + 1, 0); // size is n+1 to allow access up to index n-1
    }

    // Add value to position i
    void add(int i, int val) {
        ++i; // convert to 1-based indexing internally
        while (i <= n) {
            bit[i] += val;
            i += i & -i;
        }
    }

    // Query the prefix sum in range [0, i]
    int prefixQuery(int i) {
        ++i;
        int res = 0;
        while (i > 0) {
            res += bit[i];
            i -= i & -i;
        }
        return res;
    }

    // Get value at a single index (for point update + range query version)

```

```

int pointQuery(int i) {
    return prefixQuery(i);
}
// Perform range update: add `val` to all elements in [l, r]
void rangeAdd(int l, int r, int val) {
    add(l, val);
    add(r + 1, -val);
}
};

```

MO Algo :

```

void solve(){
    int n , q ; cin>>n>>q ;
    int arr[n] ;
    vector<array<int,3>>qi;
    vector<int>res(q) ;
    vector<int>freq(2e5+5);
    int id = 1 ;
    int BS = sqrt(n) ;

    lp(i,n)cin>>arr[i];

    lp(i,q){
        int l , r ; cin>>l>>r ;
        qi.push_back({--l,--r,i});
    }
    map<int,int>mp ;
    for (int i = 0; i < n; i++)
    {
        if(!mp[arr[i]]){
            mp[arr[i]]=id; arr[i]= id ; id++;
        }else{
            arr[i] = mp[arr[i]] ;
        }
    }

    int ans = 0 ;
    auto add = [&](int index){
        int val = arr[index] ;
        if(!freq[val])ans++ ;
        freq[val]++;
    };
    auto erase = [&](int index){
        int val = arr[index] ;
        freq[val]--;
        if(!freq[val])ans-- ;
    };
};

```

```

    sort(qi.begin(),qi.end(),[&](array<int,3>q1 , array<int,3>q2){
        return make_pair(q1[0]/BS , q1[1]) < make_pair(q2[0]/BS , q2[1]) ;
    });
    int l = qi[0][0] ;
    int r = l ;
    add(l);
    for(auto [lx , rx , ind]: qi){
        while (lx<l)add(--l);
        while (rx>r)add(++r);

        while (lx>l)erase(l++);
        while (rx<r)erase(r--);

        res[ind] = ans;
    }
    for (int i = 0; i < q; i++)cout<<res[i] ln ;
}

```

MO with RollBack

```

void solve(){
    int n , m , k ; cin>>n>>m>>k ;
    int arr[n] ;
    lp(i,n)cin>>arr[i] ;
    int B = sqrt(n)+2 ;
    vector<array<int,3>>qi [B] ;
    map<int,int>res ;
    m++;
    int tL[m] ;
    memset(tL , -1 , sizeof tL) ;
    for (int i = 0; i < k; i++)
    {
        int l , r ; cin>>l>>r ;
        l-- , r--;
        if(r-l+1<=B){
            int a = 0 ;
            for (int j = l; j <= r; j++)
            {
                int val = arr[j] ;
                if(tL[val]==-1)tL[val] = j ;
                else a = max(a,abs(j-tL[val]));
            }
            res[i] = a ;
            for (int j = l; j <= r; j++)
            {
                int val = arr[j] ;
                tL[val] = -1 ;
            }
        }
    }
}

```

```

        }else{
            qi[l/B].push_back({r,l,i});
        }

    }
    for (int i = 0; i < B; i++)
    {
        sort(qi[i].begin(),qi[i].end());
    }
    int L[m] , R[m] ;

    for (int i = 0; i < B; i++)
    {
        int ans = 0 ;
        if(qi[i].empty())continue;
        for (int i = 0; i < m; i++)L[i] = R[i] = -1 ;
        int l = B*i+(B-1) ;
        int lst = l ;
        int r = l ;
        L[arr[l]] = R[arr[l]] = 1 ;
        for (auto [rq , lq, iq ] : qi[i] )
        {
            while (rq>r)
            {
                r++;
                int val = arr[r] ;
                if(R[val]==-1){
                    L[val] = R[val] = r ;
                }else{
                    R[val] = r ;
                    ans = max(ans , abs(L[val]-R[val]));
                }
            }
            vector<array<int,3>>updates ;
            int tmp = ans ;
            while (l>lq)
            {
                l--;
                int val = arr[l] ;
                updates.push_back({L[val],R[val],val});
                if(R[val]==-1){
                    L[val] = R[val] = l ;
                }else{
                    L[val] = l ;
                    tmp = max(tmp , abs(L[val]-R[val]));
                }
            }
            res[iq] = max(tmp,ans);
            reverse(updates.begin(),updates.end());
            for(auto u : updates){
                L[u[2]] = u[0];
                R[u[2]] = u[1];
            }
            l = lst ;
        }
    }

```



```

    }
}
for (int i = 0; i < k; i++)
{
    cout<<res[i] ln ;
}
}

```

MO with DSU :

```

void solve(){
    int n , m ; cin>>n>>m ;
    vp arr(m) ;

    int B = sqrt(m)+2;
    lp(i,m){
        int a,b ; cin>>a>>b; a--;b--;
        arr[i].first = a ; arr[i].second = b;
    }
    int q ; cin>>q ;
    vi res(q) ;
    vector<array<int,3>>qi [B];
    DSU d ; d.init(n) ; d.snapshot();
    for (int i = 0; i < q; i++)
    {
        int a , b ; cin>>a>>b; a--; b--;
        if(b-a+1<=B){
            d.snapshot();
            for (int j = a; j <= b; j++)
            {
                d.merge(arr[j].first,arr[j].second);
            }
            res[i] = d.cnt ;
            d.rollback();
            continue;
        }
        qi[a/B].push_back({b,a,i});
    }
    for (int i = 0; i < B; i++)
    {
        sort(all(qi[i]));
    }
    for (int i = 0; i < B; i++)
    {
        if(qi[i].empty())continue;
        int l = min(B*i+(B-1),m-1) ;
        int r = l ;
        int tempL = l ;
        d.snapshot();
    }
}

```

```

d.merge(arr[l].first, arr[l].second);
for(auto [rx , lx , qx] : qi[i]){
    while (rx>r)
    {
        r++; d.merge(arr[r].first , arr[r].second);
    }
    d.snapshot();
    while (lx<l)
    {
        l--; d.merge(arr[l].first , arr[l].second);
    }
    res[qx] = d.cnt ;
    d.rollback();
    l = tempL ;
}
d.rollback();
}
for (int i = 0; i < q; i++)cout<<res[i] ln ;
}

```

Orderd Set :

Definition :

```

#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/assoc_container.hpp>
namespace __gnu_pbds{
    typedef tree<long long,
                null_type,
                less<long long>,
                rb_tree_tag,
                tree_order_statistics_node_update> ordered_set;
}
using namespace __gnu_pbds;

```

How to use :

define an object like `this : ordered_set st ;`
then you can deal with it as you deal with `set`

Extra Functions :

`find_by_order(k) .. O(log(N))`

It returns an iterator to the kth `element (counting from zero)` in the `set` in `O(log(n))` time.

To find the first element k must be zero.

Let us assume we have a `set s : {1, 5, 6, 17, 88}`,

then :

`*(s.find_by_order(2)) : 3rd element in the set is 6`

`*(s.find_by_order(4)) : 5th element in the set is 88`

order_of_key(k).. $O(\log(N))$ //

It returns to the number of items that are strictly smaller than our item k in $O(\log(n))$ time.

Let us assume we have a set s : {1, 5, 6, 17, 88}

then :

s.order_of_key(6) : Count of elements strictly smaller than 6 is 2.

s.order_of_key(25) : Count of elements strictly smaller than 25 is 4.

How to change it to Ordered Multiset :

less -- > less_equal

but if you want to erase an element .. you must erase it like this :

st.erase(st.upper_bound(X));

```
#include <ext/pb_ds/assoc_container.hpp>
```

```
#include <ext/pb_ds/tree_policy.hpp>
```

```
using namespace __gnu_pbds;
```

```
class OrderedSet {
```

```
private:
```

```
    typedef tree<long long,
                null_type,
                less_equal<long long>,
                rb_tree_tag,
                tree_order_statistics_node_update> ordered_set;
    ordered_set s;
```

```
public:
```

```
    void Insert(long long x) {
        s.insert(x);
    }
```

```
    bool Exist(long long x) {
        if (s.upper_bound(x) == s.end()) {
            return false;
        }
        return (*s.upper_bound(x)) == x;
    }
```

```
    int LessThan(long long x) {
        return s.order_of_key(x);
    }
```

```
    void Erase(long long x) {
        if (Exist(x)) {
            s.erase(s.upper_bound(x));
        }
    }
```

```
    int Idx(long long x) {
        if (!Exist(x)) {
            return -1;
        }
        return s.order_of_key(x);
    }
```

```

int LastIdx(long long x) {
    if (!Exist(x)) {
        return -1;
    }
    if (*s.find_by_order(s.size() - 1) == x) {
        return s.size() - 1;
    }
    return Idx(*s.lower_bound(x)) - 1;
}

long long Value(int index) {
    return *s.find_by_order(index);
}

int Count(long long x) {
    if (!Exist(x)) {
        return 0;
    }
    return LastIdx(x) - Idx(x) + 1;
}

void Clear() {
    s.clear();
}

int Size() {
    return s.size();
}

int Lower_bound(int x)
{

    return *s.upper_bound(x);
}

int Upper_bound(int x)
{

    return *s.lower_bound(x);
}
};

```

Segment tree with hashing :

```

const int M = 2 , N = 1e6 + 5;
array<int,2> B , MODS , pw [N];
struct Node
{
    int sz ;

```

```

    array<int,2>pref ;
    array<int,2>rev ;
};
struct Segtree
{
    #define L 2*node+1
    #define R 2*node+2
    #define mid ((l+r)/2)

    vector<Node> seg ;
    int sz; Node skip ;

    Node merge (Node a ,Node b){
        Node res ;
        res.sz = a.sz + b.sz ;
        for (int i = 0; i < 2; i++)
        {
            res.pref[i] = (a.pref[i]*pw[b.sz][i] + b.pref[i])%MODS[i];
        }
        for (int i = 0; i < 2; i++)
        {
            res.rev[i] = (b.rev[i]*pw[a.sz][i] + a.rev[i])%MODS[i];
        }
        return res ;
    }

    Node query (int l , int r , int node ,int lx , int rx){
        if(r<lx || rx<l)return skip ;
        if(l>=lx&&r<=rx) return seg[node];
        Node left = query (l , mid ,L , lx , rx);
        Node right= query (mid+1,r,R , lx,rx);
        return merge(left,right);
    }

    void update (int l , int r, int node , int indx , int val){
        if(l==r){
            for (int i = 0; i < 2; i++)
            {
                seg[node].pref[i] = (B[i]*val)%MODS[i];
                seg[node].rev[i] = (B[i]*val)%MODS[i];
            }
            seg[node].sz = 1;
            return ;
        }
        if(indx<=mid){
            update(l,mid,L,indx,val);
        }
        else{
            update(mid+1,r,R,indx,val);
        }
        seg[node] = merge (seg[L],seg[R]);
    }

    Segtree(int szz){
        int n = szz;
        sz = 1 ;
    }
}

```

```

    for (int i = 0; i < 2; i++)
    {
        skip.pref[i] = 0 ;
        skip.rev[i] = 0 ;
    }
    skip.sz = 0 ;
    while (sz<n) sz*=2 ;
    seg = vector<Node> (sz*2 , skip);
}

bool queryCall(int l , int r){
    Node res = query(0,sz-1,0,l,r);
    return res.pref == res.rev ;
}

void updateCall(int val , int indx){
    update(0,sz-1,0,indx,val);
}

#undef L
#undef R
#undef mid
};

```

Segment Tree for Max Freq in Range

```

struct Node {
    int val, freq;
    int prefixVal, prefixFreq;
    int suffixVal, suffixFreq;
};

struct SegmentTree {
    #define L 2 * node + 1
    #define R 2 * node + 2
    #define mid ((l + r) / 2)
    int sz;
    vector<Node> tree;
    vector<int> arr;
    Node skip = {-1, 0, -1, 0, -1, 0};

    Node merge(Node a, Node b) {
        if (a.freq == 0) return b;
        if (b.freq == 0) return a;
        Node res;
        // prefix
        res.prefixVal = a.prefixVal;
        res.prefixFreq = a.prefixFreq;
        if (a.prefixVal == b.prefixVal) {
            res.prefixFreq += b.prefixFreq;
        }
    }
}

```

```

    // suffix
    res.suffixVal = b.suffixVal;
    res.suffixFreq = b.suffixFreq;
    if (a.suffixVal == b.suffixVal) {
        res.suffixFreq += a.suffixFreq;
    }
    // default best
    res.val = a.val;
    res.freq = a.freq;
    if (b.freq > res.freq) {
        res.val = b.val;
        res.freq = b.freq;
    }
    // cross
    if (a.suffixVal == b.prefixVal) {
        int midFreq = a.suffixFreq + b.prefixFreq;
        if (midFreq > res.freq) {
            res.val = a.suffixVal;
            res.freq = midFreq;
        }
    }
    return res;
}

SegmentTree(const vector<int>& input) {
    int n = input.size();
    sz = 1;
    while (sz < n) sz *= 2;
    arr = input;
    tree.resize(sz * 2, skip);
    build(0, sz - 1, 0);
}

void build(int l, int r, int node) {
    if (l == r) {
        if (l < (int)arr.size()) {
            int v = arr[l];
            tree[node] = {v, 1, v, 1, v, 1};
        }
        return;
    }

    build(l, mid, L);
    build(mid + 1, r, R);
    tree[node] = merge(tree[L], tree[R]);
}

void update(int l, int r, int node, int idx, int val) {
    if (l == r) {
        arr[idx] = val;
        tree[node] = {val, 1, val, 1, val, 1};
        return;
    }

    if (idx <= mid)

```

```
        update(l, mid, L, idx, val);
    else
        update(mid + 1, r, R, idx, val);

    tree[node] = merge(tree[L], tree[R]);
}

Node query(int l, int r, int node, int ql, int qr) {
    if (r < ql || l > qr) return skip; // neutral
    if (ql <= l && r <= qr) return tree[node];
    Node left = query(l, mid, L, ql, qr);
    Node right = query(mid + 1, r, R, ql, qr);
    return merge(left, right);
}

void updateCall(int idx, int val) {
    update(0, sz - 1, 0, idx, val);
}

Node queryCall(int l, int r) {
    return query(0, sz - 1, 0, l, r);
}

#undef L
#undef R
#undef mid
};
```